

HTML5

Introducción

HTML, siglas de HyperText Markup Language, hace referencia al lenguaje de marcado para la elaboración de páginas web. Es un estándar que, en sus diferentes versiones, define una estructura básica y un código (denominado código HTML) para la definición de contenido de una página web, como texto, imágenes, etc. Es un estándar a cargo de la W3C, organización dedicada a la estandarización de casi todas las tecnologías ligadas a la web, sobre todo en lo referente a su escritura e interpretación.

El lenguaje HTML basa su filosofía de desarrollo en la referenciación. Para añadir un elemento externo a la página (imagen, vídeo, script, etc.), este no se incrusta directamente en el código de la página, sino que se hace una referencia a la ubicación de dicho elemento mediante texto. De este modo, la página web contiene sólo texto mientras que recae en el navegador web (interpretador del código) la tarea de unir todos los elementos y visualizar la página final. Al ser un estándar, HTML busca ser un lenguaje que permita que cualquier página web escrita en una determinada versión, pueda ser interpretada de la misma forma (estándar) por cualquier navegadores web actualizado.

Sin embargo, a lo largo de sus diferentes versiones, se han incorporado y suprimido características, con el fin de hacerlo más eficiente y facilitar el desarrollo de páginas web compatibles con distintos navegadores y plataformas (PC de escritorio, portátiles, teléfonos inteligentes, tablets, etc.). Sin embargo, para interpretar correctamente una nueva versión de HTML, los desarrolladores de navegadores web deben incorporar estos cambios y el usuario debe ser capaz de usar la nueva versión del navegador con los cambios incorporados. Usualmente los cambios son aplicados mediante parches de actualización automática (Firefox, Chrome) u ofreciendo una nueva versión del navegador con todos los cambios incorporados, en un sitio web de descarga oficial (Internet Explorer). Un navegador no actualizado no será capaz de interpretar correctamente una página web escrita en una versión de HTML superior a la que pueda interpretar, lo que obliga muchas veces a los desarrolladores a aplicar técnicas y cambios que permitan corregir problemas de visualización e incluso de interpretación de código HTML. Así mismo, las páginas escritas en una versión anterior de HTML deberían ser actualizadas o reescritas, lo que no siempre se cumple. Es por ello que ciertos navegadores aún mantienen la capacidad de interpretar páginas web de versiones HTML anteriores. Por estas razones, aún existen diferencias entre distintos navegadores y versiones al interpretar una misma página web.

Todo es una etiqueta

Las páginas HTML son en esencia archivos de texto sin formato con etiquetas añadidas que proporcionan información sobre cómo se estructura el documento. Las etiquetas van entre los símbolos ‘<’ y ‘>’. Las etiquetas determinan la forma en como el navegador tratará el texto contenido en ellas, de allí viene el nombre de lenguaje de marcado de texto.

Todas las etiquetas a excepción algunas especiales, deben abrirse y cerrarse. Para abrir una etiqueta se utiliza **<etiqueta>** y para cerrar **</etiqueta>**. Todo lo que se encuentre entre este par de elementos sufrirá los efectos de la etiqueta en cuestión.

La primer página

Hagamos ahora nuestra primera página en html, bastará con escribir lo siguiente:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi primera página</title>
</head>
<body>
```

```
<p>Hola Mundo! .</p>
</body>
</html>
```

Todas las páginas tienen dos elementos principales: cabeza y cuerpo. A continuación se describen las etiquetas utilizadas en el ejemplo anterior.

- **<head>**. Gestiona información sobre el documento y otros datos que no se consideran parte del contenido real del documento. Esto incluye el título de la página, sus meta etiquetas, hojas de estilo y cualquier script. En resumen cualquier cosa sobre o que afecte al contenido de la propia página. La información del elemento head no puede verla el usuario, sólo ve la información del elemento **body**.
- **<body>**. Es el lugar donde se encuentra todo lo que el usuario verá en el navegador. Cualquier texto, enlace o imagen que quiera mostrar en el navegador se encuentra ahí.
- **<html>**. Indica al navegador que el texto contenido dentro de la etiqueta debe ser tratado como un documento HTML.
- **<title>**. Es el título de la página que aparecerá en la ventana o pestaña del navegador. También es la forma como se identificará la página si se guarda en los favoritos del navegador.
- **<p>**. Indica que el texto contenido en ella sea presentado como un párrafo.
- **<DOCTYPE>**. Es una parte adicional, aunque importante de la página Web que indica al navegador cómo presentar la página y qué lenguaje se ha utilizado para etiquetarla. Debe ir antes de la etiqueta **<html>** inicial. Si no se especifica el tipo de documento, el navegador asume que se trata de una versión básica de HTML, por lo que si la página contiene elementos de una versión reciente de HTML, XHTML o algún otro, el contenido no se mostrará correctamente.
- **namespace**. Un segundo atributo necesario para crear páginas Web HTML Strict válidas, se conoce también como una declaración XML **namespace**. El atributo `xmlns="http://www.w3.org/1999/xhtml"` es necesario para escribir páginas Web en XHTML. Añadido a la etiqueta de inicio **<html>** sirve para validar la página.
- **Codificación de caracteres**. Se añade **charset="type"** en la etiqueta meta para indicar el tipo de codificación de caracteres utilizado en los archivos html que leerá el navegador. El más adecuado para nuestra lengua es **"utf-8"**.

En HTML5 el encabezado inicial `<!DOCTYPE html>` no lleva información adicional, en versiones anteriores se utilizaban más parámetros hacia el navegador. Esta convención la introdujo la W3C con el objetivo de simplificar el código, ahora los navegadores se encargan de detectar los parámetros que previamente se escribían de forma explícita.

Otras etiquetas útiles. ****. Énfasis fuerte, presentado por defecto en negrita en los navegadores. ****. Énfasis, presentado por defecto en cursiva. **<!-- -->**. Esta etiqueta permite escribir comentarios dentro del código HTML, por lo que el navegador ignorará todo lo que se encuentre dentro.

Una regla importante es que si se anidan etiquetas, es decir que se abre una etiqueta dentro de otra, debe cerrarse primero la última que se ha abierto. Por ejemplo:

```
<p><strong><em>Soy un párrafo, estoy fuertemente enfatizado y en cursiva.</em></strong></p>
```

Es importante cerrar en el orden correcto las etiquetas, aunque funcionalmente podríamos no notar la diferencia, conforme la página se hace más grande y agreguemos estilos en ella, ocurrirá que la página se verá diferente en distintos navegadores o incluso que no se muestre en lo absoluto.

```
<p><strong><em>Soy un párrafo escrito por un programador flojo y descuidado.</strong></em></p>
```

Como en todos los lenguajes, es importante que el programador agregue comentarios en su código. Los comentarios son útiles para el programador mismo principalmente, ya que por ejemplo podrían ayudar a recordar como se estructuró el documento, como nota para indicar cuando cambió el documento, como nota para un compañero que trabaje en la misma página o simplemente para ocultar partes de la misma página.

Ejemplo:

```
<h1>Comentarios en acción</h1>
<p>Soy el primer párrafo, me muestro en el navegador. El 1er. encabezado se muestra también.</p>
<!-- Soy un comentario, no aparezco en el navegador -->
<p>Soy el segundo párrafo, también me muestro en el navegador.</p>
<!-- <p>Soy el tercer párrafo, me han ocultado.</p> -->
```

Etiquetado Estructurado

Las etiquetas son una herramienta útil y poderosa, es importante realizar etiquetado con significado. En primer lugar nos centramos en la semántica. Antes incluso de abrir el editor de texto, observe el contenido con el que ha estado trabajando y sepárelo en las partes que lo componen. Asígnele una jerarquía y trate de sacar el significado de frases específicas. Una vez hecho esto, etiquete el documento en función de ello.

Encabezados

- **<h1>Texto muy grande</h1>**

sirve para colocar la frase que indica el título del contenido de una página web.

- **<h2>Texto grande</h2>**

indica títulos de importancia para subsecciones del documento web actual.

- **<h3>Texto algo más grande de lo normal</h3>**

- **<h4>Texto normal</h4>**

- **<h5>Texto pequeño</h5>**

- **<h6>Texto muy pequeño</h6>**

Las etiquetas h3, h4, h5 y h6 nos permiten definir títulos de subapartados de un bloque encabezado con un h2.

¿Qué es más importante, una etiqueta <h1> o una <h2>?, ¿es una <h3> más importante que una <h5>?. La respuesta es sencilla, en HTML hay seis encabezados <h1>, <h2>, <h3>, <h4>, <h5> y <h6>. <h1> es el más importante y <h6> el menos.

El contenido de la página debe estar organizado en este punto por importancia, no por como se ve el texto con los encabezados. El estilo se define posteriormente con CSS.

Texto Estructurado

La estructura del texto en un sitio es fundamental para que la información sea comprensible y usable por el usuario.

Listas

Las listas son semánticas: sugieren estructura o indican grupos de información, que es por lo que las utilizamos. Las listas proporcionan un medio para agrupar información y hacer más fácil de comprender los elementos desglosados. Las listas también nos proporcionan etiquetas con significado, a las que posteriormente podemos aplicar estilo con CSS.

Listas sin orden

Estructuramos listas sin ordenar utilizando dos elementos: **ul** y **li**. El elemento **ul** indica que agrupamos nuestros elementos en una lista sin ordenar, es decir cada elemento de la lista es de igual valor y la lista no sugiere ningún orden inherente. El elemento **li** se utiliza para cada elemento de la lista.

Ejemplo:

```
<h1>Séptima Generación de las Consolas de Videojuegos</h1>
<ul>
  <li>PlayStation 3</li>
  <li>Nintendo Wii</li>
  <li>Xbox 360</li>
</ul>
```

Los marcadores de elemento pueden cambiarse si se agrega **TYPE=tipo** a la etiqueta de apertura de la lista, sustituyendo tipo por alguno disponible. Los elementos disponibles son:

- disc
- circle
- square

Por defecto en una lista sin orden se utiliza **disc**.

Listas ordenadas

Para las listas ordenadas se utilizan los elementos **ol** y **li**. El elemento **ol** indica que agrupamos nuestros elementos en una lista ordenada y cada elemento de la lista se agrega con **li**.

Ejemplo:

```
<h1>Éxito en ventas de las Consolas de videojuegos de la Séptima Generación</h1>
<ol>
  <li>Nintendo Wii</li>
  <li>Xbox 360</li>
  <li>PlayStation 3</li>
</ol>
```

Los marcadores de elemento pueden cambiarse si se agrega **TYPE=tipo** a la etiqueta de apertura de la lista, sustituyendo tipo por alguno disponible. Los elementos disponibles son:

- TYPE=A: A, B, C, D, ...
- TYPE=a: a, b, c, d, ...

- TYPE I: I, II, III, IV, ...
- TYPE i: i, ii, iii, iv, ...

Por defecto en una lista ordenada se utilizan números arábigos.

Listas Descriptivas

Para las listas descriptivas se utilizan los elementos **dl**, **dt** y **dd**. El elemento **dl** en conjunto con **dt** indica que agrupamos nuestros elementos en una lista descriptiva y cada elemento de la lista se agrega con **dd**.

Ejemplo:

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- white cold drink</dd>
</dl>
```

Formularios

Los formularios se utilizan para enviar datos al servidor. Un formulario puede contener campos de texto, checkbox, botones de selección, botones de acción entre otros. De igual forma puede contener listas de selección, área de texto, leyendas y algunos más.

La sintaxis para crear un formulario en HTML es la siguiente:

```
<form>
  ...
  elementos de entrada
  ...
</form>
```

El elemento más importante es el de entrada **<input>**, que se utiliza para seleccionar información del usuario. El elemento puede variar de muchas formas, de acuerdo al atributo **"type"**. Puede ser un campo de texto, checkbox, contraseña entre otros.

1. Campos de texto

```
<form>
  First name: <input type="text" name="firstname"><br>
  Last name: <input type="text" name="lastname">
</form>
```

Nota: El formulario no es visible por si mismo y el ancho por defecto de un campo de texto es de 20 caracteres.

2. Campo de contraseña

```
<form>
  Password: <input type="password" name="pwd">
</form>
```

Nota: Los caracteres en el campo de contraseña se muestran como asteriscos.

3. Botones de selección

<input type="radio"> define un selector tipo radio. Estos botones permiten al usuario seleccionar solamente una de un número finito de opciones.

```
<form>
  <input type="radio" name="sex" value="male">Male<br>
  <input type="radio" name="sex" value="female">Female
</form>
```

4. Checkbox

<input type="checkbox"> define un selector tipo checkbox. Estos botones permiten al usuario seleccionar desde ninguno hasta todas las opciones de un número finito de opciones.

```
<form>
  <input type="checkbox" name="vehicle" value="Bike">I have a bike<br>
  <input type="checkbox" name="vehicle" value="Car">I have a car
</form>
```

5. Botón de acción

<input type="submit"> define un botón de enviar.

El botón **enviar** se utiliza para enviar información a un servidor. Los datos se envían a una página especificada en el atributo **action**. El archivo definido en el atributo **action** típicamente hace algo con los datos recibidos.

```
<form name="input" action="html_form_action.asp" method="get">
  Username: <input type="text" name="user">
  <input type="submit" value="Submit">
</form>
```

Tablas

Las tablas se definen con la etiqueta **<table>**.

Una tabla se divide en renglones mediante la etiqueta **<tr>**. Los renglones pueden dividirse a su vez en columnas con la etiqueta **<td>**. Un renglón puede dividirse en encabezados mediante la etiqueta **<th>**.

Los elementos **<td>** son contenedores de datos en la tabla y pueden contener todo tipo de elementos HTML como texto, imágenes, listas, otras tablas, etc.

El ancho de una tabla y formato en general puede definirse mediante CSS.

Ejemplo 1. Mi primera tabla.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Tabla</title>
</head>
```

```

<body>
  <table>
    <tr>
      <td>Elemento 1</td>
      <td>Elemento 2</td>
      <td>Elemento 3</td>
    </tr>
    <tr>
      <td>Elemento 4</td>
      <td>Elemento 5</td>
      <td>Elemento 6</td>
    </tr>
    <tr>
      <td>Elemento 7</td>
      <td>Elemento 8</td>
      <td>Elemento 9</td>
    </tr>
  </table>
</body>
</html>

```

Este ejemplo genera una tabla sin estilo, el código HTML solo debe tener el contenido de las tablas cualquier característica de estilo se hace mediante CSS. Algunas propiedades para el estilo en tablas son las siguientes:

- **border.** Añadir bordes a la tabla, definir ancho del borde y color.
- **border-collapse.** Colapsar los bordes en una sola línea.
- **padding.** Espacio que habrá entre las celdas y su contenido.

Ejemplo 2. Estilo en tablas.

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    table,th,td {
      border:1px solid black;
      border-collapse: collapse;
      padding: 15px;
    }
  </style>
</head>
<body>
  <table>
    <tr>
      <td>Elemento 1</td>
      <td>Elemento 2</td>
      <td>Elemento 3</td>
    </tr>
    <tr>
      <td>Elemento 4</td>
      <td>Elemento 5</td>
      <td>Elemento 6</td>
    </tr>
  </table>
</body>
</html>

```

```

</tr>
<tr>
  <td>Elemento 7</td>
  <td>Elemento 8</td>
  <td>Elemento 9</td>
</tr>
</table>
</body>
</html>

```

Si no se especifica el tamaño de la fuente, por defecto se establece **16px** para el texto normal, como párrafos (16px = 1em).

Para evitar el problema de redimensionar con versiones antiguas de Internet Explorer, se recomienda utilizar **em** en lugar de pixeles. La unidad de medida em es recomendada por la W3C.

em es equivalente al tamaño actual del tamaño de letra. El tamaño por defecto de la letra en los navegadores es 16px. Por lo tanto el tamaño por defecto de **em** es **16px**.

El tamaño puede ser calculado de pixels a **em** mediante la fórmula: em = pixels/16

Ejemplo 3. Tabla con más estilo.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <style>
    /* Estilo en toda la tabla */
    #consolas {
      font-family: "Trebuchet MS", Arial, Helvetica, sans-serif;
      width: 100%;
      border-collapse: collapse;
      text-align: center;
    }
    #consolas td, #consolas th {
      font-size: 1em;
      border: 1px solid #98bf21;
      padding: 3px 7px 2px 7px;
    }
    #consolas th {
      font-size: 1.1em;
      text-align: center;
      padding-top: 5px;
      padding-bottom: 4px;
      background-color: #A7C942;
      color: #ffffff;
    }
    #consolas tr.renglon1 td {
      color: #000000;
      background-color: #EAF2D3;
    }
    #consolas tr.renglon2 td {
      color: #000000;
      background-color: white;
    }
  </style>

```

```

</style>
</head>
<body>
  <table id="consolas">
    <tr>
      <th>Generación</th>
      <th>Nintendo</th>
      <th>SEGA</th>
      <th>Sony</th>
      <th>Microsoft</th>
    </tr>
    <tr class="renglon1">
      <td>3</td>
      <td>NES</td>
      <td>Master System</td>
      <td></td>
      <td></td>
    </tr>
    <tr class="renglon2">
      <td>4</td>
      <td>Super NES</td>
      <td>Genesis</td>
      <td></td>
      <td></td>
    </tr>
    <tr class="renglon1">
      <td>5</td>
      <td>Nintendo 64</td>
      <td>Saturn</td>
      <td>Playstation</td>
      <td></td>
    </tr>
    <tr class="renglon2">
      <td>6</td>
      <td>Game Cube</td>
      <td>Dreamcast</td>
      <td>Playstation 2</td>
      <td>XB0X</td>
    </tr>
    <tr class="renglon1">
      <td>7</td>
      <td>Wii</td>
      <td></td>
      <td>Playstation 3</td>
      <td>XB0X 360</td>
    </tr>
    <tr class="renglon2">
      <td>8</td>
      <td>WiiU</td>
      <td></td>
      <td>Playstation 4</td>
      <td>XB0X One</td>
    </tr>
  </table>
</body>

```

```
</html>
```

Con todo lo anterior, podemos crear un esqueleto en HTML que pueda contener cualquier tipo de página. A partir de este esqueleto podemos construir un sitio solo llenando los espacios con la información que corresponda.

Enlaces HTML

Los enlaces se encuentran en casi todas las páginas Web, los enlaces permiten al usuario ir de una página a otra.

Los enlaces en HTML son hipervínculos. Un hipervínculo es un texto o imagen donde puede darse un click para desplazarse a otro documento. En HTML los enlaces se definen con la etiqueta `<a>`.

Sintaxis:

```
<a href="url">Texto del Enlace</a>
```

Ejemplo:

```
<a href="http://lab.anahuac.mx/~hselley/ppi/">PPI</a>
```

El atributo `href` especifica la dirección destino del enlace. El `Texto del Enlace` es la parte visible.

Los valores que puede tomar el atributo `href` son los siguientes:

Valor	Ejemplo
URL Absoluta	<code>http://ejemplo.com/</code>
URL Relativa	<code>href="index.html"</code>
Enlace Interno	<code>href="#inicio"</code>
Otros Protocolos	<code>http://, ftp://, mailto:, file:, etc.</code>
Un script	<code>href="javascript:alert('Hola mundo');"</code>

Colores en los enlaces

Cuando el usuario pasa el cursor sobre un enlace, dos cosas normalmente pasan:

- El cursor cambia de acuerdo al navegador y sistema operativo.
- El texto del enlace cambiará de color.

Por defecto, el color del texto de un enlace aparece de la siguiente forma:

- Un enlace no visitado está subrayado y en azul.
- Un enlace visitado está subrayado y en púrpura.
- Un enlace activo está subrayado y en rojo.

Se pueden cambiar los colores por defecto con CSS.

```
<style>
a:link {
    color:green;
    background-color:transparent;
}
```

```

    text-decoration:none;
}
a:visited {
    color:pink;
    background-color:transparent;
    text-decoration:none;
}
a:hover {
    color:red;
    background-color:transparent;
    text-decoration:underline;
}
a:active {
    color:yellow;
    background-color:transparent;
    text-decoration:underline;
}
}
</style>

```

El atributo target en los enlaces HTML

El atributo target especifica donde y como se abrirá el documento enlazado. Los valores posibles que puede tomar el atributo son los siguientes:

Valor	Descripción
_blank	Abre el enlace en una nueva ventana o pestaña.
_self	Abre el enlace en el contenedor del enlace.
_parent	Abre el enlace en el contenedor padre.
_top	Abre el enlace en el cuerpo completo de la ventana.
framename	Abre el enlace en el contenedor especificado en el valor del atributo.

Imágenes como enlaces

En HTML es posible utilizar imágenes como enlaces, por ejemplo:

```

<a href="pagina.html">
  
</a>

```

Enlaces Internos

Los enlaces internos se utilizan para que los lectores puedan ir a partes específicas de una página Web. Para crear un enlace interno basta con asignar un id a alguna etiqueta específica, luego añadir un enlace hacia ella. De esta forma, cuando el enlace es presionado la página se desplazará hacia la ubicación de la etiqueta con ese id.

```

<h2 id="tips">Sección de Tips</h2>
<a href="#tips">Visite la Sección de Tips Útiles</a>

```

También es posible añadir un enlace interno de un documento enlazado.

```
<a href="html_tips.html#tips">Tips</a>
```

Imágenes en HTML

En HTML las imágenes se definen con la etiqueta `<img>`. La etiqueta solo contiene atributos y no tiene una etiqueta de cerrado. El atributo `src` especifica la dirección (URL o local) de la imagen.

Sintaxis:

```

```

El atributo `alt` especifica un texto alternativo a la imagen si esta no puede ser desplegada por alguna razón. Aunque este atributo no es necesario si es muy recomendable utilizarlo para que una página se valide correctamente.

Adicionalmente, este atributo lo utilizan los lectores de pantalla para personas con alguna discapacidad visual.

Tamaño de las imágenes

Para especificar el tamaño en que será desplegada una imagen se puede hacer desde el atributo `style`. Dentro de este atributo se puede definir el ancho y alto (`width` y `height` respectivamente) con el cual la imagen será desplegada por el navegador. Los valores se especifican en pixeles.

```

```

También pueden utilizarse los atributos `width` y `height` directamente en la etiqueta `<img>`.

```

```

Aunque ambos metodos para establecer el tamaño de la imagen son válidos para el estándar HTML5, se recomienda utilizar el atributo `style`. Esto debido a que de esta forma se evita que alguna hoja de estilos externa o interna cambie el tamaño de la imagen (técnicamente estamos usando un estilo en línea).

Esqueleto de una página en HTML5

Con todo lo anterior, podemos crear un esqueleto en HTML que pueda contener cualquier tipo de página. A partir de este esqueleto podemos construir un sitio solo llenando los espacios con la información que corresponda.

Ejemplo 4. Esqueleto de una página en HTML 5

HTML

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Esqueleto de una página</title>
  <link rel="stylesheet" type="text/css" href="css/estiloMacho.css">
  <script type="text/javascript" src="js/script.js"></script>
```

```

</head>
<body>
  <div id="contenedor">
    <!-- Dividir la página en varias secciones -->
    <div id='top'>
      <!-- Logotipo superior -->
      <div id='topLogo'>
        <a href='http://www.w3.org/'>
          <img src='src/HTML5_Logo_512.png' alt='google.com' height='64'
style='border:0;'>
        </a>
      </div>
    </div>
    <div id='topnav'>
      <!-- Navegación superior -->
      <div>
        <a class='topnav' href='home.html'>HOME</a>
        <a class='topnav' href='Lugar1.html'>Lugar 1</a>
        <a class='topnav' href='Lugar2.html'>Lugar 2</a>
        <a class='topnav' href='Lugar3.html'>Lugar 3</a>
        <a class='topnav' href='Mas.html'>Más...</a>
        <div class='topNavRight'>
          <a class='topnav' href='Ejemplos.html'> EJEMPLOS</a>|
          <a class='topnav' href='foro.html'> FORO</a>|
          <a class='topnav' href='acerca.html'> ACERCA DE</a>
        </div>
      </div>
    </div>
    <div id="cabecera">
      <!-- Cabecera del sitio -->
      <h1>Cabecera del sitio</h1>
    </div>
    <div id="menu">
      <!-- Menú del costado izquierdo -->
      <h4>Índice del contenido de la página</h4>
      <ul>
        <li><a href="#cabecera">Inicio</a></li>
        <li><a href="#seccion1">Sección 1</a></li>
        <li><a href="#seccion2">Sección 2</a></li>
        <li><a href="#seccion3">Sección 3</a></li>
        <li><a href="#seccion4">Sección 4</a></li>
      </ul>
    </div>
    <div id="contenido">
      <div id="principal">
        <!-- Contenido principal de la página -->
        <h2>Acá va el contenido principal de la página.</h2>

        <!-- Sección 1 del Contenido -->
        <h3 id="seccion1">Sección 1</h3>
        <p>Contenido Seccion 1</p>

        <!-- Sección 2 del Contenido -->
        <h3 id="seccion2">Sección 2</h3>
        <p>Contenido Seccion 2</p>
      </div>
    </div>
  </div>

```

```

        <hr>
    </div>

    <div id="secundario">
        <!-- Contenido secundario de la página -->
        <h2>Contenido secundario.</h2>
        <p>Aquí pueden ir enlaces o imágenes complementarias a la información
principal.</p>
        <p>Esta sección es opcional, depende del diseño que se elija.</p>
        <p>Google ofrece una gran variedad de fuentes para los desarrolladores Web en
<a href="https://www.google.com/fonts#">Google Fonts</a></p>
    </div>
</div>
<div id="pie">
    <!-- Pié de página -->
    Aquí van enlaces externos o internos para navegar el sitio.<br>
</div>
</div>
</body>
</html>

```

CSS

```

html {
    overflow-y:scroll;
}
body {
    font-family: sans-serif;
    font-size:0.8125em;
    color:#404040;
    background-color:#F6F4F0;
    margin:0px;
}
body,p,h1,h2,h3,h4,table,td,th,ul,ol,textarea,input {
    font-family: verdana,Helvetica,arial,sans-serif;
}
h1,h2,h3,h4,h5,h6 {
    background-color:transparent;
    color:#000000;
}
h1 {
    font-size:28px;
    margin-top:0px;
    font-weight:normal;
}
h2 {
    font-size:22px;
    margin-top:10px;
    margin-bottom:10px;
    font-weight:normal;
}
h3 {
    font-size:17px;
    font-weight:normal;
}

```

```
h4 {
    font-size:12px;
}
h5 {
    font-size:11px;
}
h6 {
    font-size:10px;
}
#menu {
    position: fixed;
    float: left;
    width: 10%;
    height: 500px;
}
#topnav {
    height:25px;
    background-color:#d3d3d3;
}
.topNavRight {
    float:right;
    word-spacing:0;
}
a:link{
    color: blue;
}
a:hover{
    color: red;
}
#cabecera {
    width: 100%;
    margin:auto;
    height:10%;
    text-align: center;
}
#contenido {
    float: right;
    width: 90%;
    height: 100%;
}
#contenido #principal {
    background-color: #F4F4F4;
    float: left;
    width: 80%;
    height: 90%;
}
#contenido #secundario {
    float: left;
    width: 20%;
    height: 90%;
    font-style: bold;
    border-style: none;
    text-align: center;
}
#pie {
```

```
clear: both;
font-style: italic;
text-align: center;
height: 10%;
}
```

Multimedia

Multimedia en la Web es el sonido, musica, videos, peliculas y animaciones.

Los contenidos multimedia se almacenan en archivos cuya extensión nos dice el tipo de contenido multimedia.

Video

Antes de HTML5 no existía un estandar para los videos para las páginas en Internet, los vídeos solo se reproducen con un plugin. La etiqueta en HTML5 video permite añadir un video en una página web.

HTML5 solamente soporta de forma nativa video en los formatos siguientes formatos:

- MP4
- WebM
- Ogg

Ejemplo 5. Videos en HTML5

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Video</title>
</head>
<body>
  <video>
    <source src="sw3.mp4" type="video/mp4">
    Su navegador no soporta el elemento de audio.
  </video>
</body>
</html>
```

Existen más atributos opcionales de la etiqueta video.

Audio

Al igual que con el video, antes de HTML5 no existía un estandar para reproducir audio para las páginas en Internet, los archivos de audio solo se reproducen con un plugin. La etiqueta en HTML5 audio permite añadir un audio en una página web.

HTML5 solamente soporta de forma nativa audio en los formatos siguientes formatos:

- MP3
- WAV
- Ogg

Ejemplo 6. Audio en HTML5

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Audio</title>
</head>
<body>
  <audio>
    <source src="QJ.mp3" type="audio/mpeg">
    Su navegador no soporta el elemento de audio.
  </audio>
</body>
</html>

```

Plugins

El propósito de los plugins en HTML5 es extender la funcionalidad de los navegadores en HTML. Los plugins también son conocidos como applets Java. Los plugins pueden ser añadidos en las páginas web con las etiquetas <object> o <embed>.

Los plugins pueden ser utilizados para muchos propósitos como desplegar mapas, buscar virus, verificar su identidad en la banca electrónica, etc.

La etiqueta <object> es soportada por todos los navegadores, define un objeto incrustado en un documento HTML.

Ejemplo 7. Incrustar objetos a una página con la etiqueta <object>.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo de Plugins</title>
</head>
<body>
  <h1>Ejemplo de Plugins</h1>
  <h2>SWF</h2>
  <object data="src/inicio.swf" type=""></object>

  <h2>Imágenes</h2>
  <object data="src/cap.png" type=""></object>

</body>
</html>

```

La etiqueta <embed> es soportado por la mayoría de los navegadores. Esta etiqueta también permite incrustar objetos dentro de los documentos HTML.

Ejemplo 8. Incrustar objetos a una página con la etiqueta <embed>.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">

```

```

    <title>Ejemplo de Plugins</title>
</head>
<body>
    <h1>Ejemplo de Plugins</h1>
    <h2>SWF</h2>
    <embed src="src/inicio.swf">

    <h2>Imágenes</h2>
    <embed src="src/cap.png">
</body>
</html>

```

YouTube

HTML5 permite la reproducción de videos de forma nativa sin la necesidad de algún plugin adicional, sin embargo solo soporta algunos formatos de videos. Es posible convertir los videos, pero requiere software especial para conversión y tiempo para llevarla a cabo. La forma más sencilla de reproducir videos en HTML es a través de YouTube.

Actualmente YouTube proporciona el código HTML necesario para incrustar un video, este se encuentra en la sección compartir.

Ejemplo 9. Incrustar un video de YouTube a una página HTML.

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejemplo de Video</title>
</head>
<body>
    <h1>Ejemplo de Video incrustado desde YouTube</h1>
    <iframe width="420" height="315" src="https://www.youtube.com/embed/z9UzlicjwrM"
        frameborder="0" allowfullscreen></iframe>
</body>
</html>

```

La etiqueta <iframe> tiene los siguientes parámetros.

- **autoplay**

- **Valor 0:** Los controles del reproductor son visibles.
- **Valor 1:** Los controles del reproductor se ocultan cuando se reproduce el video.
- **Valor 2 (por defecto):** Si el reproductor tiene relación de aspecto 16:9 o 4:3 toma el valor de 1, cualquier otro caso toma el valor de 0.

- **controls**

- **Valor 0 (por defecto):** El video no se reproduce automáticamente cuando el reproductor sea cargado.
 - **Valor 1:** El video se reproducirá automáticamente cuando el reproductor sea cargado.
- **controls**
 - **Valor 0:** Los controles del reproductor son ocultos. El video se carga inmediatamente.

- **Valor 1 (por defecto):** Los controles del reproductor son visibles. El video se carga inmediatamente.
- **Valor 2:** Los controles del reproductor son visibles pero el video no se carga inmediatamente.
- **loop**
 - **Valor 0 (por defecto):** El video se reproduce una sola vez.
 - **Valor 1:** El video se reproducirá indefinidamente.
- **playlist**
 - Una lista de videos separados por coma.

Geolocalización en HTML5

La API de geolocalización en HTML se utiliza para determinar la localización de un usuario.

Probar

Utilizando Geolocalización en HTML

El método `getCurrentPosition()` es utilizado para obtener la posición del usuario.

Ejemplo 10. Obtener las coordenadas del usuario.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Geolocalización</title>
</head>
<body>
  <p>Presione el botón para obtener su ubicación.</p>
  <button onclick="getLocation()">Presionar</button>
  <p id="demo"></p>
  <script>
    var x = document.getElementById("demo");

    function getLocation() {
      if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition);
      } else {
        x.innerHTML = "Su navegador no soporta la geolocalización.";
      }
    }
    function showPosition(position) {
      x.innerHTML = "Latitud: " + position.coords.latitude +
        "<br>Longitud: " + position.coords.longitude;
    }
  </script>
</body>
</html>
```

Este ejemplo funciona de la siguiente forma:

- El `if` verifica si el navegador soporta la geolocalización.

- Si el navegador soporta la geolocalización se ejecuta el método `getCurrentPosition()`, si no se despliega un mensaje de error.
- Si el método `getCurrentPosition()` se ejecuta correctamente se devuelve un objeto tipo coordenada a la función que se especificó como parámetro.
- La función `showPosition()` devuelve la Latitud y Longitud.

Manejo de Errores

El método `getCurrentPosition()` tiene un segundo parámetro que se utiliza para manejar posibles errores al obtener la geolocalización del usuario. Los posibles errores son los siguientes:

- `PERMISSION_DENIED`
- `POSITION_UNAVAILABLE`
- `TIMEOUT`
- `UNKNOWN_ERROR`

Ejemplo 11. Manejo de errores en la geolocalización.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Geolocalización</title>
</head>
<body>
  <p>Presione el botón para obtener su ubicación.</p>
  <button onclick="getLocation()">Presionar</button>
  <p id="demo"></p>

  <script>
    var x = document.getElementById("demo");

    function getLocation() {
      if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition, showError);
      } else {
        x.innerHTML = "Su navegador no soporta la geolocalización.";
      }
    }

    function showPosition(position) {
      x.innerHTML = "Latitude: " + position.coords.latitude +
        "<br>Longitude: " + position.coords.longitude;
    }

    function showError(error) {
      switch(error.code) {
        case error.PERMISSION_DENIED:
          x.innerHTML = "El usuario declinó la petición de Geolocalización."
          break;
        case error.POSITION_UNAVAILABLE:
          x.innerHTML = "La información de geolocalización no está
disponible."
          break;
        case error.TIMEOUT:
          x.innerHTML = "La petición del usuario rebasó el tiempo de espera."
```

```

        break;
    case error.UNKNOWN_ERROR:
        x.innerHTML = "Error desconocido."
        break;
    }
}
</script>
</body>
</html>

```

Desplegar el resultado en un mapa

Para desplegar el resultado en un mapa, se necesita acceder a un servicio de mapas, como puede ser Google Maps.

Ejemplo 12. Mostrar la ubicación del usuario en Google Maps (una imagen estática).

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>Geolocalización</title>
    <script>
        var x = document.getElementById("demo");

        function getLocation() {
            if (navigator.geolocation) {
                navigator.geolocation.getCurrentPosition(showPosition, showError);
            } else {
                x.innerHTML = "Su navegador no soporta geolocalización.";
            }
        }

        function showPosition(position) {
            var latlon = position.coords.latitude + "," + position.coords.longitude;

            var img_url = "http://maps.googleapis.com/maps/api/staticmap?center="
            +latlon+"&zoom=14&size=400x300&sensor=false";
            document.getElementById("mapholder").innerHTML = "<img src='"+img_url+"'>";
        }

        function showError(error) {
            switch(error.code) {
                case error.PERMISSION_DENIED:
                    x.innerHTML = "El usuario declinó la petición de Geolocalización."
                    break;
                case error.POSITION_UNAVAILABLE:
                    x.innerHTML = "La información de geolocalización no está
disponible."
                    break;
                case error.TIMEOUT:
                    x.innerHTML = "La petición del usuario rebasó el tiempo de espera."
                    break;
                case error.UNKNOWN_ERROR:
                    x.innerHTML = "Error desconocido."
                    break;
            }
        }
    </script>
</head>
<body>
    <div id="demo">
        <div id="mapholder">
            <img alt="Mapa de Google Maps que muestra la ubicación del usuario." data-bbox="113 337 879 911"/>
        </div>
    </div>
</body>
</html>

```

```

    }
  }
</script>
</head>
<body>
  <p id="demo">Presione el botón para obtener su ubicación.</p>
  <button onclick="getLocation()">Presionar</button>
  <div id="mapholder"></div>
</body>
</html>

```

En este ejemplo se hace una petición a la API de Google Maps, pasándole como parámetro las coordenadas obtenidas, y la API devuelve una imagen del mapa correspondiente a las coordenadas. La petición se hace mediante: `http://maps.googleapis.com/maps/api/staticmap?center="+latlon+"&zoom=14&size=400x300&sensor=false`. Observe que la variable `latlon` contiene las coordenadas obtenidas.

Desplegar el resultado en un objeto mapa.

Ejemplo 13. Mostrar la ubicación del usuario en un objeto de Google Maps.

```

<!DOCTYPE html>
<html>
<head>
  <title>Geolocalización</title>
  <meta charset="utf-8">
  <script src="http://maps.google.com/maps/api/js?sensor=false"></script>
  <script>
    var x = document.getElementById("demo");
    function getLocation() {
      if (navigator.geolocation) {
        navigator.geolocation.getCurrentPosition(showPosition, showError);
      } else {
        x.innerHTML = "Su navegador no soporta geolocalización.";
      }
    }

    function showPosition(position) {
      lat = position.coords.latitude;
      lon = position.coords.longitude;
      latlon = new google.maps.LatLng(lat, lon)
      mapholder = document.getElementById('mapholder')
      mapholder.style.height = '250px';
      mapholder.style.width = '500px';

      var myOptions = {
        center:latlon,zoom:14,
        mapTypeId:google.maps.MapTypeId.ROADMAP,
        mapTypeControl:false,
        navigationControlOptions: {
          style:google.maps.NavigationControlStyle.SMALL
        }
      }

      var map = new google.maps.Map(document.getElementById("mapholder"),

```

```

myOptions);
    var marker = new google.maps.Marker({
        position:latlon,
        map:map,
        title:"¡Aquí esta!"
    });
}

function showError(error) {
    switch(error.code) {
        case error.PERMISSION_DENIED:
            x.innerHTML = "El usuario declinó la petición de Geolocalización."
            break;
        case error.POSITION_UNAVAILABLE:
            x.innerHTML = "La información de geolocalización no está
disponible."
            break;
        case error.TIMEOUT:
            x.innerHTML = "La petición del usuario rebasó el tiempo de espera."
            break;
        case error.UNKNOWN_ERROR:
            x.innerHTML = "Error desconocido."
            break;
    }
}
</script>
</head>
<body>
    <p id="demo">Presione el botón para obtener su ubicación.</p>
    <button onclick="getLocation()">Presionar</button>
    <div id="mapholder"></div>
</body>
</html>

```

Si desea más información acerca de la API de Google Maps puede consultarla aquí.